

1.3 Listen und Rekursion

Die Taxizentrale führt eine Liste der Taxis im Dienst. Taxis kommen dazu und melden sich ab – in beliebiger Reihenfolge. Eine Warteschlange nimmt aber nur vorne heraus. Unsere **Liste** kennt nur ihren ersten Knoten (anfang), alles Weitere erledigen die Knoten. *Website: Station 1.3, Projekt Taxizentrale*

Aufgabe 1 Wie viele stehen hinter dir?

Die erste Person in der Mensa-Schlange will wissen, wie viele anstehen. Jede darf nur ihren Hintermann fragen.

a) Ergänze die Regel:

„Steht hinter mir niemand, antworte ich 1. Sonst frage ich meinen Hintermann und antworte mit seiner Antwort + 1.“

b) Spielt es zu fünft durch. Wer antwortet zuerst, wer zuletzt?

zuerst die letzte Person (1),

zuletzt die erste Person (5)

Aufgabe 2 Projekt Taxizentrale, TODO 2

a) Links zählt `anzahl()` aus 1.1 mit einer Wiederholung. Rechts arbeitet jeder Knoten nach der Regel aus Aufgabe 1. Ergänze die Lücken und beschrifte die Kommentare mit *Abbruchbedingung* und *rekursiver Aufruf*.

iterativ (Warteschlange, 1.1)

```
public int anzahl() {
    int zaehler = 0;
    Knoten aktuell = anfang;
    while (aktuell != null) {
        zaehler++;
        aktuell = aktuell.getNachfolger();
    }
    return zaehler;
}
```

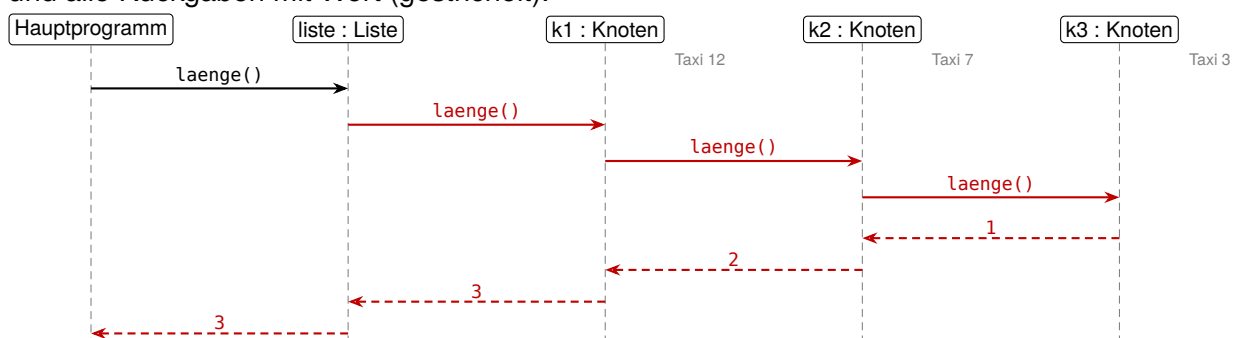
rekursiv (Knoten)

```
public int laenge() {
    if (nachfolger == null) { // Abbruchbedingung
        return 1;
    } else {
        return nachfolger.laenge() + 1; // rekursiver Aufruf
    }
}
```

rekursiv (Liste)

```
public int laenge() {
    if (anfang == null) { return 0; }
    else { return anfang.laenge(); }
}
```

b) Ergänze das Sequenzdiagramm für `liste.laenge()` bei drei Knoten: alle Aufrufe (durchgezogen) und alle Rückgaben mit Wert (gestrichelt).



Rekursion

Eine Methode heißt rekursiv, wenn sie sich selbst aufruft – hier für den Nachfolger. Sie braucht eine Abbruchbedingung, sonst würde sie sich endlos weiter aufrufen. Die Liste ist eine **rekursive Datenstruktur**: Ein Knoten mit seinem Rest ist wieder eine Liste. Darum folgen auch die Methoden der Struktur: Jeder Knoten erledigt seinen Teil und gibt den Rest des Auftrags an seinen Nachfolger weiter.

Aufgabe 3 Projekt Taxizentrale, TODO 3

```
// Knoten: hängt d ans Ende an
public boolean add(DatenElement d) {
    if (nachfolger == null) {
        nachfolger = new Knoten(d);
        return true;
    } else {
        return nachfolger.add(d);
    }
}

// Liste
public boolean add(DatenElement d) {
    if (anfang == null) { // Sonderfall
        anfang = new Knoten(d);
        return true;
    } else {
        return anfang.add(d);
    }
}
```

a) Ergänze add und übertrage es ins Projekt.

b) Der letzte Knoten liefert true. Was geschieht mit diesem Wert auf dem Rückweg?

Jeder Knoten gibt ihn mit return

unverändert weiter bis zur Liste.

c) Warum braucht die Liste einen Sonderfall, der Knoten aber nicht?

In der leeren Liste ist anfang null:

Es gibt keinen Knoten, der fragen könnte.

Aufgabe 4 Projekt Taxizentrale, TODO 4a bis 4c

a) Gleich oder dasselbe? Sage die Ausgabe vorher:

```
Taxi a = new Taxi(7, "Aylin");
Taxi b = new Taxi(7, "Aylin");
println(a == b);
```

false: zwei verschiedene Objekte

== prüft, ob zwei Verweise auf dasselbe Objekt zeigen. Für inhaltliche Gleichheit erweitern wir den Vertrag DatenElement (4a):

boolean istGleich(DatenElement d);

b) Danach meldet der Compiler einen Fehler in Taxi. Warum? Ergänze dort (4b):

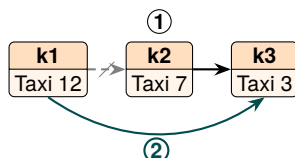
```
// Taxi (4b)
public boolean istGleich(DatenElement d) {
    return gibSchluesselwert().equals(
        d.gibSchluesselwert()) ;
}

// Knoten (4c): Ist d ab hier enthalten?
public boolean enthaelt(DatenElement d) {
    if (daten.istGleich(d)) {
        return true;
    } else if (nachfolger == null) {
        return false;
    } else {
        return nachfolger.enthaelt(d);
    }
}
```

b) Taxi erfüllt den erweiterten Vertrag noch nicht.

Aufgabe 5 Projekt Taxizentrale, TODO 5

Taxi 7 meldet sich ab. Ein Knoten kann sich nicht selbst aushängen: Den Verweis auf ihn hat sein Vorgänger. Deshalb schaut jeder Knoten auf seinen Nachfolger voraus.



- ① k1 prüft die Daten seines Nachfolgers.
② Passen sie, überspringt k1 ihn.

```
// Knoten
public boolean entfernen(DatenElement d) {
    if (nachfolger == null) {
        return false;
    } else if (nachfolger.getDaten().istGleich(d)) { // ①
        nachfolger = nachfolger.getNachfolger(); // ②
        return true;
    } else {
        return nachfolger.entfernen(d);
    }
}
```

Den ersten Knoten kann niemand überspringen. Sonderfall in Liste.entfernen:

```
if (anfang.getDaten().istGleich(d))
    anfang = anfang.getNachfolger() ;
```

Es geht auch anders – besprecht mündlich, warum man es so machen kann:

- laenge() ginge auch mit einer Wiederholung wie anzahl(). Was ist leichter zu lesen, was braucht weniger Speicher?
- Die Liste könnte wie die Warteschlange einen Verweis auf das ende haben. Was würde add dann sparen?
- entfernen im Knoten könnte statt boolean den Knoten zurückgeben, der danach an seiner Stelle steht (sich selbst oder seinen Nachfolger). Was müsste der Aufrufer mit der Rückgabe tun?